

Git & VS Code Setup Guide for Beginners

Prerequisites

- Git installed on your computer
- VS Code installed and ready to use

Step 1: Check and Configure Git (One-time Setup)

First, Check If Git Is Already Configured

Open your terminal (Command Prompt, PowerShell, or Terminal) and check your current Git configuration:

```
git config --global user.name
git config --global user.email
```

If these commands return your name and email:

- You're already set up! Skip to Step 2.

If these commands return nothing or incorrect information:

- Continue with the configuration below.

Configure Git

```
git config --global user.name "Your Full Name"
git config --global user.email "your.email@example.com"
```

Important: Use your real name and email address. This information will be attached to your commits.

Verify Your Configuration

```
git config --global --list
```

This shows all your global Git settings.

Optional: Set Your Default Editor

```
git config --global core.editor "code --wait"
```

This sets VS Code as your default Git editor.

Step 2: Create Your First Repository

Method 1: Start a New Project

1. Create a new folder for your project
2. Open VS Code: File → Open Folder → Select your project folder
3. Open VS Code terminal: `Ctrl + `` (backtick) or View → Terminal
4. Initialize Git repository:

```
git init
```

Method 2: Clone an Existing Repository

```
git clone https://github.com/username/repository-name.git
```

Step 3: Understanding Git Workflow

The Three States of Files

1. **Working Directory** - Files you're currently editing
2. **Staging Area** - Files ready to be committed
3. **Repository** - Committed files (saved snapshots)

Basic Workflow

```
Working Directory → Staging Area → Repository  
(git add)           (git commit)
```

Step 4: Making Your First Commit

1. **Create a file** (e.g., `README.md` , `index.html`)
2. **Check status:**

```
git status
```

3. **Add file to staging:**

```
git add README.md  
# Or add all files:  
git add .
```

4. **Commit your changes:**

```
git commit -m "Initial commit: Add README file"
```

Step 5: Using VS Code's Built-in Git Features

Source Control Panel

- Click the branch icon () in the left sidebar
- See all modified files at a glance
- View differences by clicking on files

Making Commits in VS Code

1. **Stage changes:** Click the `+` button next to files
2. **Write commit message** in the text box
3. **Commit:** Click the checkmark (✓) button or press `Ctrl+Enter`

VS Code Git Indicators

- **Green U:** Untracked (new) file
- **Yellow M:** Modified file
- **Red D:** Deleted file
- **Blue numbers:** Show added/removed lines

Step 6: Essential Git Commands

Checking Status and History

```
git status          # See current state of files
git log             # View commit history
git log --oneline   # Condensed commit history
```

Adding and Committing

```
git add filename.txt  # Stage specific file
git add .             # Stage all changes
git add *.js          # Stage all JavaScript files

git commit -m "Descriptive message about changes"
git commit -am "Add and commit in one step (tracked files only)"
```

Viewing Changes

```
git diff            # See unstaged changes
git diff --staged   # See staged changes
git show            # Show last commit details
```

Undoing Changes

```
git checkout -- filename.txt  # Discard changes to file
git reset HEAD filename.txt   # Unstage file
git reset --soft HEAD~1       # Undo last commit (keep changes)
```

Step 7: Good Commit Message Practices

Format

Brief summary (50 characters or less)

More detailed explanation if needed. Wrap at 72 characters.

Explain what changed and why, not how.

Examples of Good Commit Messages

- Add user authentication feature
- Fix bug in password validation
- Update README with installation instructions
- Refactor database connection logic

Examples of Bad Commit Messages

- fix
- update stuff
- asdfgh
- final version

Step 8: Working with Branches (Optional for Beginners)

Basic Branch Commands

```
git branch          # List all branches
git branch feature-name # Create new branch
git checkout feature-name # Switch to branch
git checkout -b feature-name # Create and switch to branch
git merge feature-name  # Merge branch into current branch
```

Step 9: Connecting to GitHub (Optional)

Add Remote Repository

```
git remote add origin https://github.com/username/repository-name.git
```

Push to GitHub

```
git push -u origin main    # First push
git push                   # Subsequent pushes
```

Common Beginner Mistakes to Avoid

1. **Don't commit everything:** Use `.gitignore` for files you don't want to track
2. **Write meaningful commit messages:** Future you will thank present you
3. **Commit often:** Small, frequent commits are better than large ones
4. **Don't panic:** Almost everything in Git can be undone
5. **Check status before committing:** Always run `git status` first

Quick Reference Cheat Sheet

Command	Description
<code>git init</code>	Initialize new repository
<code>git status</code>	Check file status
<code>git add .</code>	Stage all changes
<code>git commit -m "message"</code>	Commit staged changes
<code>git log</code>	View commit history
<code>git diff</code>	See unstaged changes
<code>git push</code>	Upload to remote repository
<code>git pull</code>	Download from remote repository

Next Steps

Once comfortable with these basics, explore:

- Branching and merging
- Remote repositories (GitHub, GitLab)
- Resolving merge conflicts
- Git hooks
- Advanced Git commands

Remember: Git has a learning curve, but these basics will handle 80% of your daily Git needs!